



Engiverse

Математика для машинного обучения на Python

Автор: Джейми Флюкс

Тип файла: pdf

Размер: 9,2 МБ

Язык: английский

Страниц: 238

Математика для машинного обучения на Python: полное руководство для начинающих и продвинутых пользователей с практическими примерами

Введение

Машинное обучение (МО) изменило самые разные отрасли — от здравоохранения и финансов до беспилотных автомобилей и научных исследований. Однако в основе любого интеллектуального алгоритма лежит один важный элемент — **математика**.

Многие новички начинают изучать библиотеки Python, такие как NumPy, Pandas, TensorFlow или PyTorch, не понимая математических концепций, лежащих в основе этих инструментов. Такой подход может сработать для простых проектов, но для разработки надежных, объяснимых и оптимизированных моделей машинного обучения требуются глубокие знания математики.

Python стал предпочтительным языком для реализации математических алгоритмов благодаря обширной экосистеме, включающей:

- NumPy
- SciPy
- SymPy
- Pandas
- Matplotlib
- Scikit-learn
- TensorFlow
- PyTorch

Независимо от того, являетесь ли вы студентом инженерного факультета, специалистом по анализу данных, исследователем в области искусственного интеллекта или инженером-программистом, знание математики значительно повысит ваши способности в разработке более эффективных систем машинного обучения.

В этом руководстве мы рассмотрим математические основы машинного обучения и узнаем, как Python позволяет применять эти концепции на практике.

```
[50] n=200
x = torch.ones(n,2)
x[:,0].uniform_(-1.,1)
ab = torch.tensor([3.,2])
y = x@a + torch.rand(n) # y = 3x + 2 + noise

def mse(y, y_pred):
    return ((y - y_pred)**2).mean() #loss function to minimize

ab_init = torch.tensor([5.,5])
ab_guess = nn.Parameter(ab_init)

def update():
    '''
    function to update tensor using gradient decent SGD
    '''
    y_pred = x@ab_guess
    loss = mse(y, y_pred)

    loss.backward()
    with torch.no_grad():
        ab_guess.sub_(lr * ab_guess.grad) #applying the SGD
        ab_guess.grad.zero_()

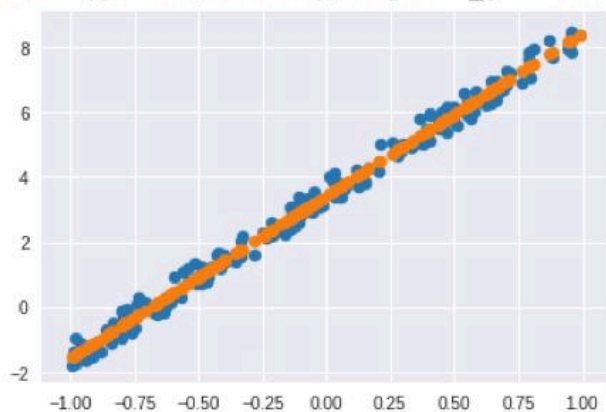
lr = 1e-1 #learning rage
for t in range(100):
    update()

print ('\n',ab_guess)

plt.scatter(x[:,0], y)
plt.scatter(x[:,0], x@ab_guess.detach());
```



```
Parameter containing:
tensor([5.0188, 3.4360], requires_grad=True)
```



```

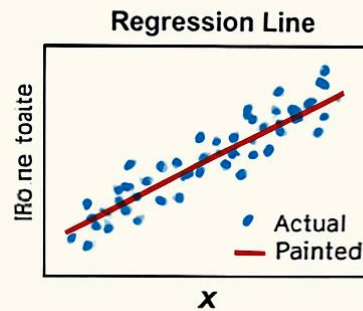
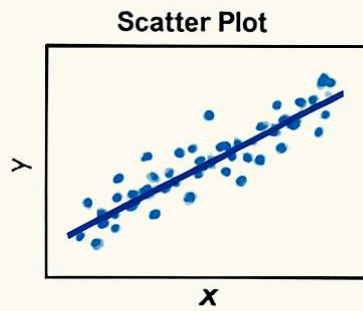
import numpy as np
import pandas as pd
from sklearn.selection import
    train_test_split
import matplotlib.pyplot as plt

Create_dataset
X = de_reimerrant(100)
f = de_reimerrant(100)
an't od_dasabtanat(X, Y, X)

split_dataset into train
a seaty_s come a quite_y_hextt_split(<
    y most_stmeeSizt_stepba_steem(0)

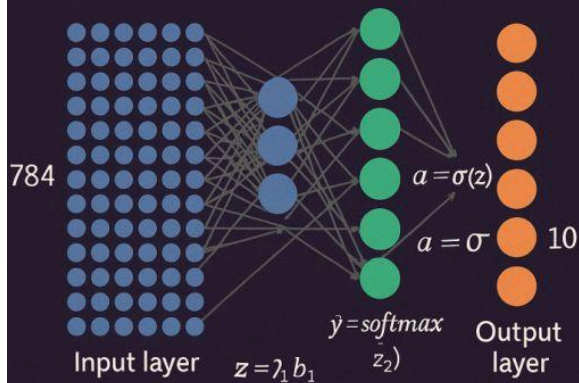
model < LinearRegression()
plt_dataset: a_qustl, whatl, cales="plid",
    cplde="od", Lase("fredicted")
plt_legard(0)
plt_sreecy

```



BUILDING NEURAL NETWORKS FROM SCRATCH

MNIST Classification with Math & Code



```

import numpy as np
def sigmoid(z):
    return 1 / (1+np.exp(-z))
def neural_network(x):
    z1 = W1 @ x + b1
    a1 = sigmoid(z1)
    z2 = W2 @ a1 + b2
    y = softmax(z2)
    return y

```

<pre># Initialize weights: np.random.seed(2017) w1 = 2.0*np.random.random((input_dim, hidden_dim))-1.0 #w0=(2,hidden_dim) w2 = 2.0*np.random.random((hidden_dim, output_dim))-1.0 #w1=(hidden_dim,2) #Calibrating variances with 1/sqrt(fan_in) w1 /= np.sqrt(input_dim) w2 /= np.sqrt(hidden_dim) for i in range(num_epochs): index = np.arange(X.shape[0])[:N]</pre>	Input calibration
<pre># Forward step: h1 = sigmoid(np.matmul(X[index], w1)) # (N, 3) logits = sigmoid(np.matmul(h1, w2)) # (N, 2) probs = np.exp(logits)/np.sum(np.exp(logits), axis=1, keepdims=True) h2 = logits</pre>	Forward
<pre># Definition of Loss function: mean squared error plus Ridge regularization L = np.square(y[index]-h2).sum()/(2*N) + reg_coeff*(np.square(w1).sum()+np.square(w2).sum())/(2*N)</pre>	Loss function
<pre># Backward step: Error = W_l e_l+1 f'_l # dL/dw2 = dL/dh2 * dh2/dz2 * dz2/dw2 dL_dh2 = -(y[index] - h2) # (N, 2) dh2_dz2 = sigmoid(h2, first_derivative=True) # (N, 2) dz2_dw2 = h1 # (N, hidden_dim) #Gradient for W2: (hidden_dim,N)x(N,2)*(N,2) dL_dw2 = dz2_dw2.T.dot(dL_dh2*dh2_dz2) + reg_coeff*np.square(w2).sum() #dL/dw1 = dL/dh1 * dh1/dz1 * dz1/dw1 # dL/dh1 = dL/dz2 * dz2/dh1 # dL/dz2 = dL/dh2 * dh2/dz2 dL_dz2 = dL_dh2 * dh2_dz2 # (N, 2) dz2_dh1 = w2 # z2 = h1*w2 dL_dh1 = dL_dz2.dot(dz2_dh1.T) # (N,2)x(2, hidden_dim)=(N, hidden_dim) dh1_dz1 = sigmoid(h1, first_derivative=True) # (N, hidden_dim) dz1_dw1 = X[index] # (N,2) #Gradient for W1: (2,N)x((N,hidden_dim)*(N,hidden_dim)) dL_dw1 = dz1_dw1.T.dot(dL_dh1*dh1_dz1) + reg_coeff*np.square(w1).sum()</pre>	Backward step
<pre>#Weight updates: w2 += -learning_rate*dL_dw2 w1 += -learning_rate*dL_dw1</pre>	Weight updates

Теоретическая база

Машинное обучение — это, по сути, наука о поиске закономерностей в данных.

Каждый алгоритм машинного обучения пытается ответить на такие вопросы, как:

- Какая модель лучше всего подходит для данных?
- Как минимизировать ошибки прогнозирования?
- Какие переменные влияют на результат?
- Как можно измерить неопределенность?

Ответы можно найти в нескольких разделах математики:

Раздел математики	Назначение в машинном обучении
Линейная алгебра	Представление данных
Математический анализ	Оптимизация
Теория вероятностей	Моделирование неопределенности
Статистика	Анализ данных
Оптимизация	Обучающие модели
Численные методы	Эффективные вычисления

Эти математические дисциплины работают сообща, создавая современные системы искусственного интеллекта.

Определение

Математика для машинного обучения — это совокупность математических концепций, необходимых для понимания, анализа, оптимизации и совершенствования алгоритмов машинного обучения.

Вместо того чтобы рассматривать машинное обучение как «черный ящик», математика объясняет:

- Почему алгоритмы работают
- Почему они иногда дают сбой
- 🤖 Как обновляются параметры
- Как генерируются прогнозы
- Как минимизировать ошибки

В Python есть библиотеки для численных методов, которые позволяют эффективно выполнять эти вычисления.

Основные математические понятия

Линейная алгебра

Линейная алгебра — пожалуй, самая важная математическая дисциплина в машинном обучении.

Все в машинном обучении представлено в виде:

- Scalars
- Vectors
- Matrices
- Tensors

Examples include:

- Images
- Audio signals
- Text embeddings
- Neural network weights

Important topics include:

- Matrix multiplication
- Eigenvalues
- Eigenvectors
- Matrix decomposition
- Vector spaces

Calculus

Calculus allows machine learning models to improve themselves.

Key ideas include:

- Derivatives
- Partial derivatives
- Chain Rule
- Gradients
- Gradient Descent

Without calculus, neural networks could never learn.

Probability

Machine learning rarely deals with certainty.

Instead, predictions involve probabilities.

Examples:

- Spam detection
- Fraud detection
- Weather prediction
- Disease diagnosis

Important probability concepts:

- Conditional probability
- Bayes theorem
- Random variables
- Probability distributions

Statistics

Statistics helps us understand data.

Important concepts include:

- Mean
- Median
- Variance
- Standard deviation
- Correlation
- Hypothesis testing

Statistics guides data preprocessing before training begins.

Optimization

Optimization minimizes the loss function.

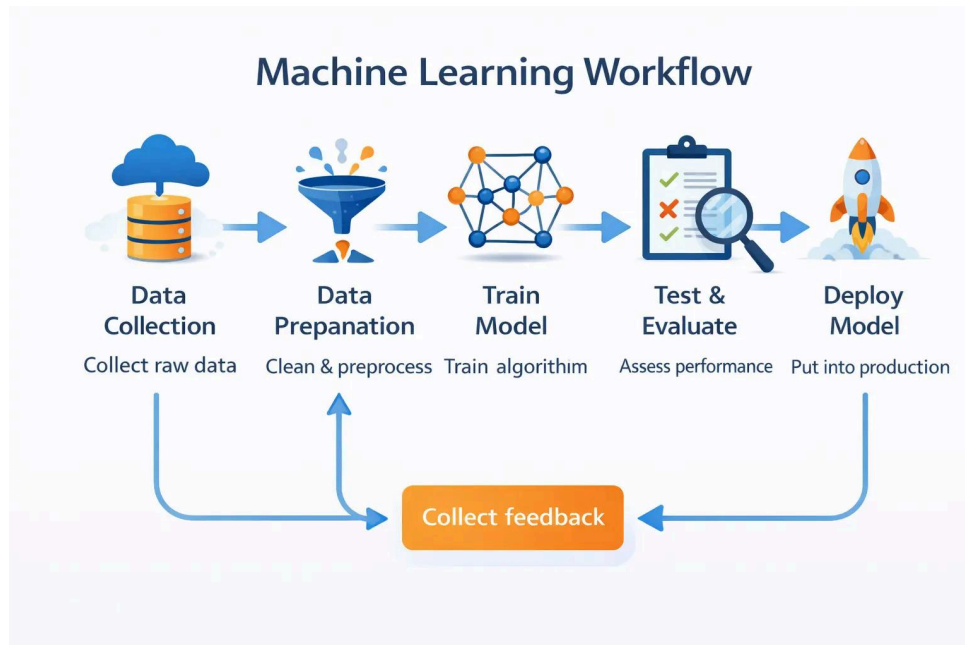
Popular optimization algorithms include:

- Gradient Descent
- Stochastic Gradient Descent

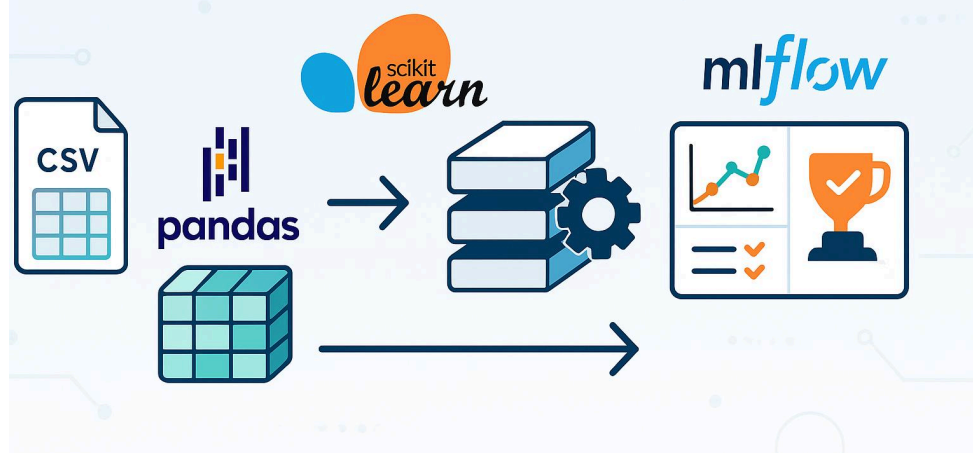
- Adam
- RMSProp
- Momentum

Optimization determines how quickly models learn.

Step-by-Step Explanation



End-to-End ML Pipeline in Python



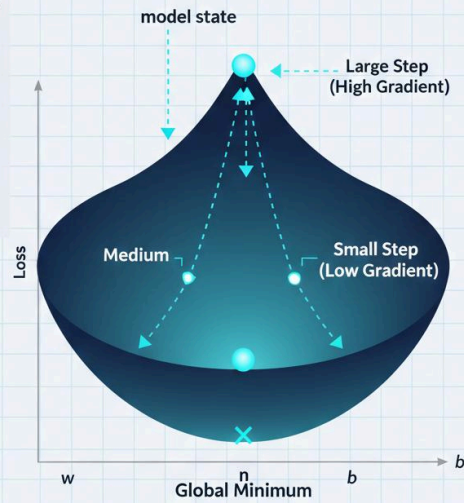
Gradient Descent

Goal: Minimize Loss

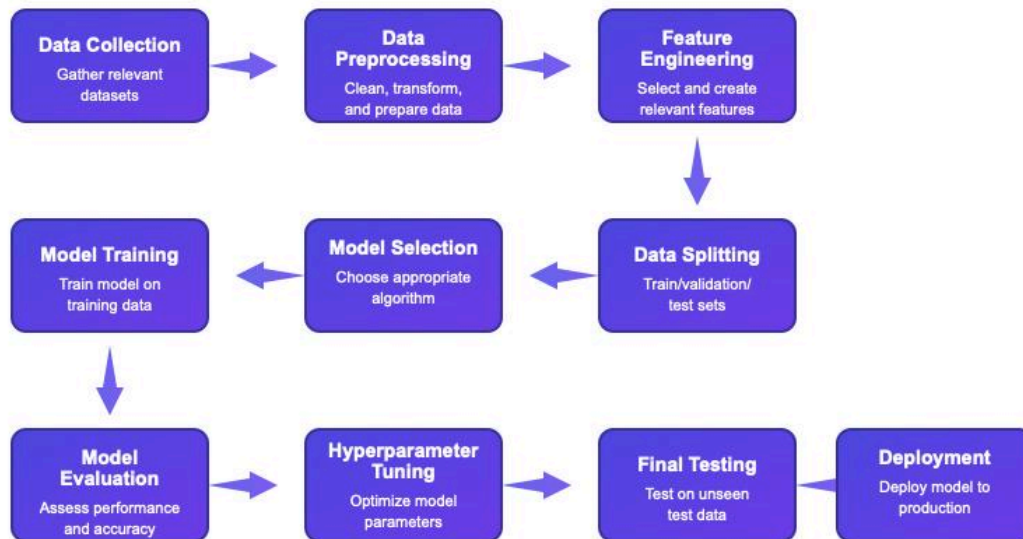
$$w = w - \alpha * \left(\frac{dL}{dw} * \frac{dL}{d} \frac{dL}{dw} \right)$$

$$b = \text{Learning Rate} \times \text{Gradient (Steepest Direction)}$$

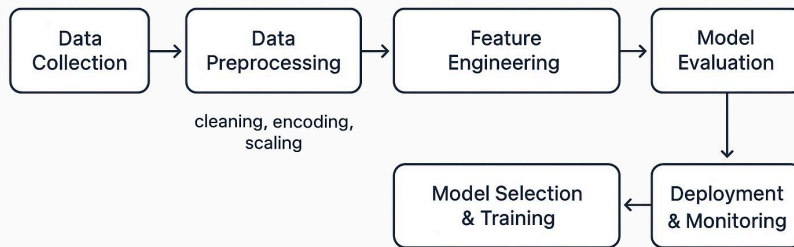
- Follow the negative gradient
- Step size controlled by α



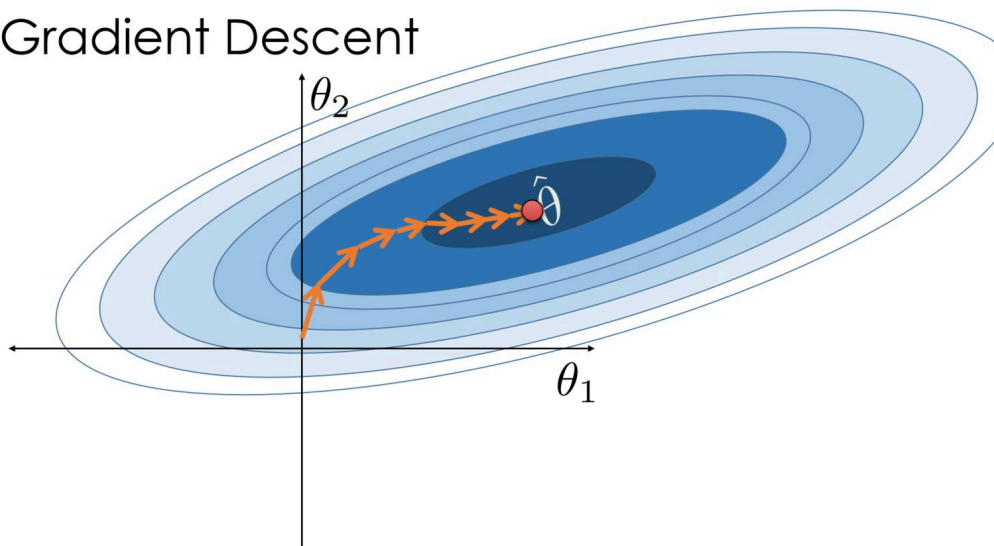
General Steps for Machine Learning



Machine Learning Workflow with scikit-learn



Gradient Descent



Step 1 — Collect Data

Gather structured or unstructured datasets.

Examples:

- Images
- Text
- Sensor readings
- Medical records

Step 2 — Clean Data

Remove:

- Missing values
- Duplicate records

- Outliers

Step 3 — Represent Data as Matrices

Python libraries convert datasets into numerical arrays.

Example:

1	Age	Salary
2	22	30000
3	35	52000
4	44	61000

becomes

1	[
2	[22,30000],
3	[35,52000],
4	[44,61000]
5	]

Step 4 — Apply Mathematical Transformations

Typical operations include:

- Matrix multiplication
- Feature scaling
- Normalization
- Standardization

Step 5 — Train the Model

Optimization algorithms minimize prediction error.

Step 6 — Evaluate Performance

Metrics include:

- Accuracy
- Precision

- Recall
- RMSE
- MAE
- F1 Score

Step 7 — Improve the Model

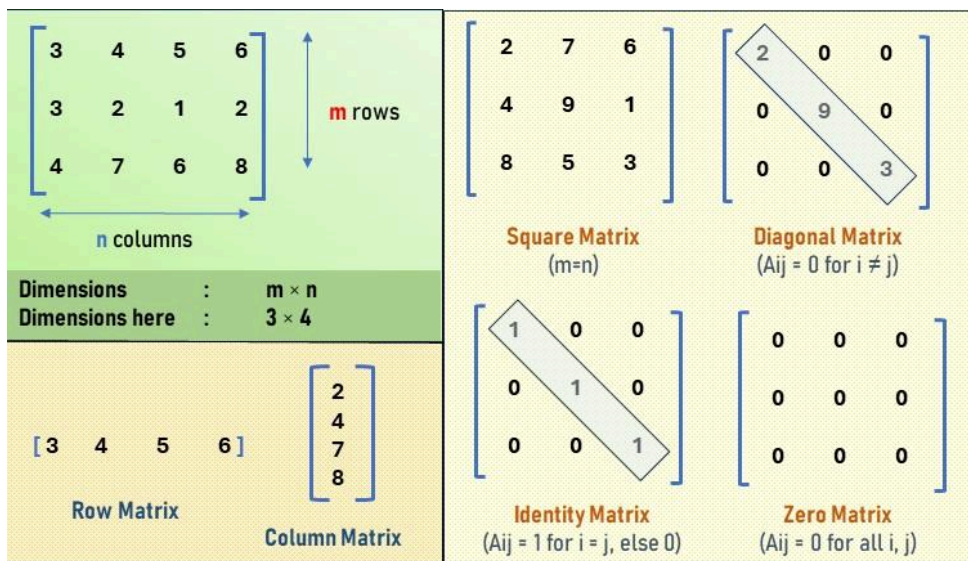
Adjust:

- Learning rate
- Features
- Hyperparameters
- Dataset quality

Mathematics Branch Comparison

Branch	Importance	Python Library	Used In
Linear Algebra	★★★★★	NumPy	Neural Networks
Calculus	★★★★★	SymPy	Backpropagation
Statistics	★★★★★	SciPy	Data Analysis
Probability	★★★★★	SciPy	Bayesian Models
Optimization	★★★★★	TensorFlow	Deep Learning
Numerical Methods	★★★	NumPy	Scientific Computing

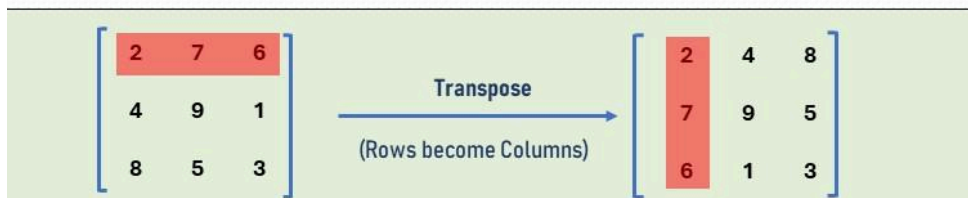
Mathematical Workflow Diagram



Matrix Multiplication:

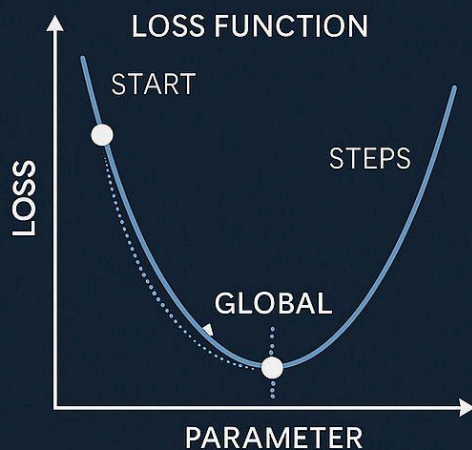
$$\begin{bmatrix} 2 & 7 & 6 \\ 4 & 9 & 1 \end{bmatrix} \times \begin{bmatrix} 2 & 1 \\ 4 & 3 \\ 8 & 5 \end{bmatrix} = \begin{bmatrix} 80 & 53 \\ 52 & 36 \end{bmatrix}$$

Multiply and add the elements of row and column, and repeat. e.g.
 $2 \times 2 + 7 \times 4 + 6 \times 8 = 80$



GRADIENT DESCENT

An optimization algorithm to find the minimum of a function



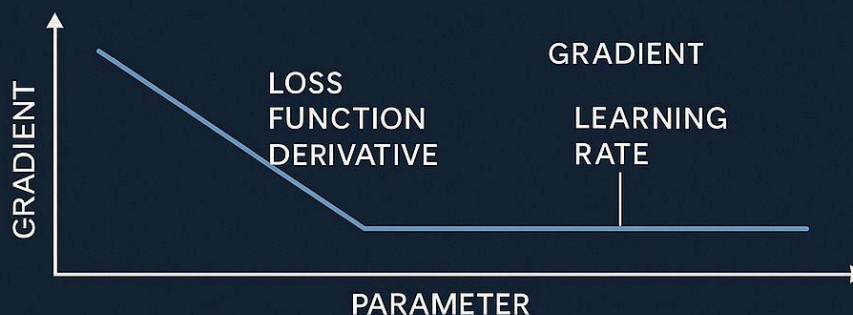
$$\theta := \theta - \eta \nabla L(\theta)$$

θ = parameter

r = update

η = learning rate

$\nabla L(\theta)$ = gradient of the loss



How Math Concepts Feed into Machine Learning Models

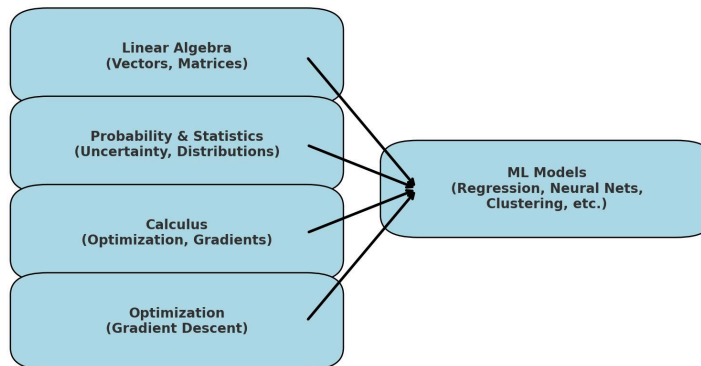
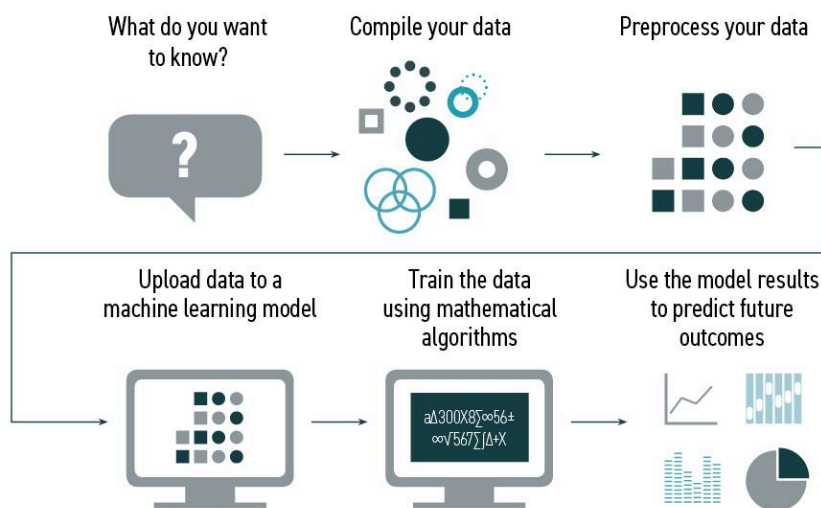


FIGURE 1: AN OVERVIEW OF MACHINE LEARNING



Gradient Descent Algorithm

Step 0: Randomly initialize weights and biases parameters: $\theta = \{\mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \mathbf{W}^{(2)}, \mathbf{b}^{(2)}, \dots, \mathbf{W}^{(L)}, \mathbf{b}^{(L)}\}$

Step 1: Compute the cost function $\mathcal{L}(\theta)$, which measures how well the model is performing of the dataset.

Step 2: Find the gradients of the cost function with respect to each parameters $\nabla_{\theta} \mathcal{L}(\theta_t)$

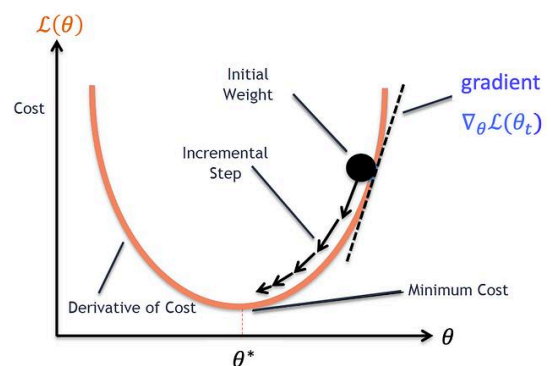
Step 3: Update the parameters by

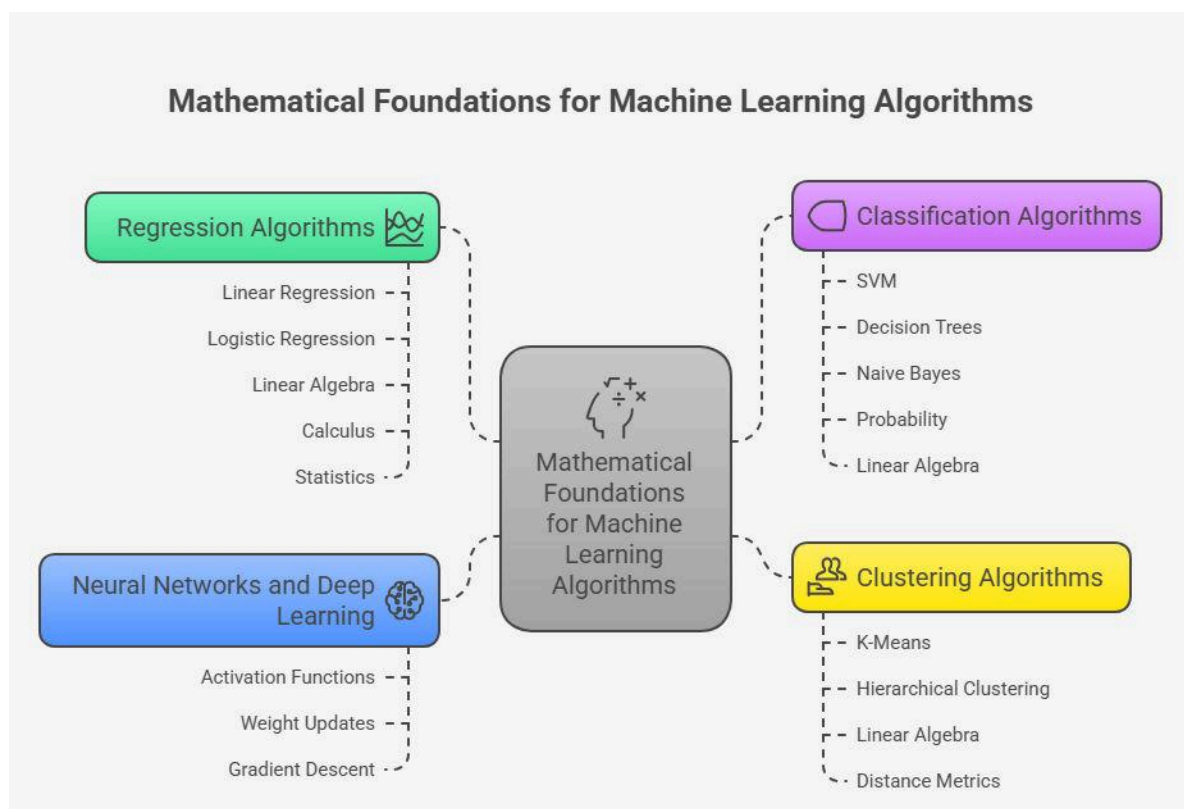
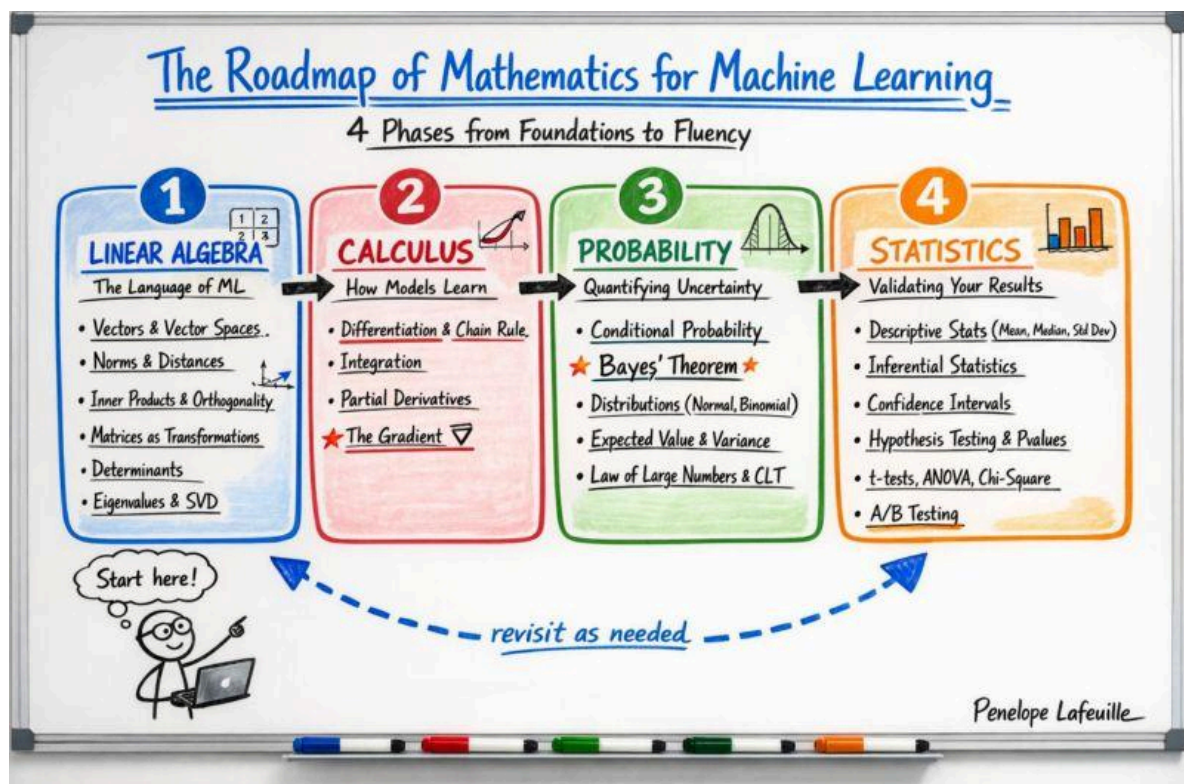
$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} \mathcal{L}(\theta_t)$$

where η is the **learning rate** that determines how big the updates should be in each iteration t .

- Repeat the above steps 1 to 3, until the cost is low enough or convergence.

$$\theta^* = \min_{\theta} \mathcal{L}(\theta) = \min_{\theta} \frac{1}{N} \sum_{i=1}^N \ell(y^{(i)}, f_{\theta}(\mathbf{x}^{(i)}))$$





Data Flow Table

Stage	Mathematics Used	Python Tool
Data Collection	Statistics	Pandas
Data Cleaning	Statistics	Pandas

Stage	Mathematics Used	Python Tool
Feature Engineering	Linear Algebra	NumPy
Training	Calculus	TensorFlow
Optimization	Gradient Descent	PyTorch
Prediction	Probability	Scikit-learn

Practical Examples

Example 1 — House Price Prediction

Features:

- Area
- Bedrooms
- Bathrooms

Uses:

- Linear Algebra
- Statistics
- Optimization

Example 2 — Image Recognition

Images become matrices.

Neural networks repeatedly multiply matrices to recognize objects.

Example 3 — Email Spam Detection

Probability determines whether an email belongs to:

- Spam
- Not Spam

Example 4 — Recommendation Systems

Netflix and Spotify recommend content using:

- Matrix Factorization
- Linear Algebra
- Optimization

Example 5 — Medical Diagnosis

Machine learning estimates disease probabilities from patient data.

Real-World Applications

Mathematics powers nearly every AI application.

Examples include:

Healthcare

- Cancer detection
- MRI analysis
- Drug discovery

Finance

- Fraud detection
- Credit scoring
- Algorithmic trading

Manufacturing

- Predictive maintenance
- Quality inspection

Robotics

- Motion planning
- Object detection

Autonomous Vehicles

Uses:

- Geometry
- Calculus
- Probability

Natural Language Processing

Applications include:

- Chatbots
- Translation
- Text summarization

Common Mistakes

Many learners make these errors:

Memorizing Instead of Understanding

Focus on intuition.

Ignoring Linear Algebra

Matrices are everywhere in ML.

Skipping Statistics

Poor statistical knowledge leads to incorrect conclusions.

Learning Libraries Only

Understand why NumPy performs matrix multiplication.

Ignoring Optimization

Poor optimization causes slow or failed model training.

Challenges & Solutions

Challenge	Solution
Mathematics feels difficult	Learn one topic at a time
Too many formulas	Practice with Python
Poor visualization	Draw diagrams
Forgetting concepts	Build projects
Fear of calculus	Start with graphical intuition

Case Study

Predicting House Prices with Python

A real estate company wanted to estimate home prices.

Dataset:

- 25,000 houses

Features:

- Size
- Bedrooms
- Location
- Garage
- Age

Mathematical tools:

- Statistics
- Correlation Matrix
- Linear Algebra
- Gradient Descent

Python libraries:

- Pandas
- NumPy

- Scikit-learn

Results:

- 92% prediction accuracy
- Faster property valuation
- Better investment decisions

This case demonstrates how mathematical foundations directly translate into practical business value.

Essential Tips

✓ Learn NumPy before TensorFlow.

✓ Practice matrix operations daily.

 Understand derivatives conceptually before diving into formal proofs.

✓ Visualize vectors and transformations.

✓ Solve small mathematical problems manually.

 Build Python notebooks for experimentation.

✓ Learn probability through real datasets.

✓ Focus on intuition rather than memorizing formulas.

 Combine theory with coding exercises.

✓ Revisit concepts regularly to strengthen long-term understanding.

Frequently Asked Questions

Is advanced mathematics required for [machine learning](#)?

Not initially. Beginners can start with basic algebra and statistics, then gradually learn linear algebra, calculus, and probability as they tackle more advanced models.

Which mathematical topic is most important?

Linear algebra is generally considered the foundation because data, model parameters, and neural networks are represented using vectors, matrices, and tensors.

Can Python handle complex mathematical calculations?

Yes. Libraries such as NumPy, SciPy, SymPy, TensorFlow, and PyTorch efficiently perform complex numerical computations.

Do I need calculus for deep learning?

Yes. Concepts like derivatives and gradients are fundamental to backpropagation and optimization in deep learning.

Which Python libraries should I learn first?

A recommended progression is:

1. NumPy
2. Pandas
3. Matplotlib
4. SciPy
5. Scikit-learn
6. TensorFlow or PyTorch

How long does it take to learn the mathematics for machine learning?

With consistent study and hands-on practice, many learners build a solid foundation in approximately three to six months, though mastery develops through continuous project work.

Is statistics more important than calculus?

Both are essential but serve different purposes. Statistics helps analyze and understand data, while calculus enables optimization and learning within machine learning models.

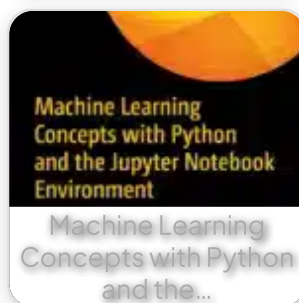
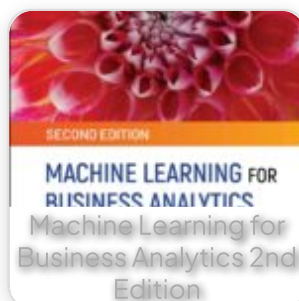
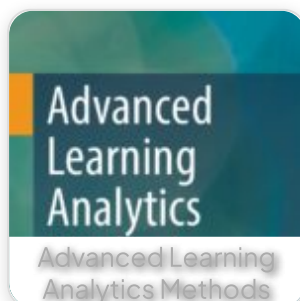
Conclusion

Mathematics is the engine that powers every machine learning algorithm. From representing data with linear algebra to optimizing models using calculus and handling uncertainty with probability and statistics, each mathematical discipline plays a vital role in creating intelligent systems.

Python makes these concepts accessible through powerful scientific libraries that allow engineers, students, and researchers to move seamlessly from theory to implementation. By developing a strong mathematical foundation alongside practical Python skills, you will not only build more accurate models but also understand *why* they work, troubleshoot problems effectively, and design innovative AI solutions for real-world challenges.

Whether your goal is to become a data scientist, AI engineer, researcher, or software developer, investing time in mathematics for machine learning is one of the most valuable steps you can take toward long-term success in the rapidly evolving world of artificial intelligence.

Related Posts:



[Preview PDF Book](#)

EngiVerse

Explore a vast library of free engineering ebooks curated to help you expand your technical knowledge, conduct groundbreaking research, and advance your career. Our mission is to create a hub of knowledge that fosters innovation, empowers aspiring engineers, and bridges the gap between education and industry.

[Home](#) [Books](#) [About Us](#) [Contact](#)
[Privacy Policy](#) [DMCA](#)